



Build **fast**. Build **right**.

Composable framework for building advanced applications with stability and flexibility.



Romuald Brillout

- Creator of Vike and Telefunc (tRPC alternative)
- Half German/French, grew up in Paris
- Computer Science at KIT (Karlsruhe, Germany)
- Lives in Munich

Intro
●○○○

Hooks
○○○○○○

Architecture
○○

Use cases
○○○

DX
○○○○○○○○

Business model
○

Future
○

Conclusion
○



Vite

webpack alternative



Vike

Next.js/Nuxt alternative

Intro
○●○○

Hooks
○○○○○○○

Architecture
○○

Use cases
○○○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
○

Status quo

Frontend frameworks:

- Tightly coupled with UI framework (e.g. Next.js $\Leftrightarrow$ React, Nuxt $\Leftrightarrow$ Vue)
- Impose their priorities (e.g. Next.js Server Components)
- 100% "free" $\Rightarrow$ skewed interests

Unstable $\Rightarrow$ but stability is crucial for large projects:

- Shouldn't be forced complex refactors
- Should adopt new technology at their own pace
- Should be able to adopt new technology progressively

Vike

A fundamentally new approach.

Intro
○○●○

Hooks
○○○○○○○

Architecture
○○

Use cases
○○○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
○



vite-plugin-ssr

Like Next.js/Nuxt but as do-one-thing-do-it-well Vite plugin.

- Started vite-plugin-ssr in 2021
 - Vite's early days (before its meteoric rise)
 - Idea: Powerful low-level hooks ⇒ full control over integration
- Renamed to Vike
 - For "street cred"
 - Extensions: vike-react / vike-vue / vike-solid (and more)

```
// pages/hello/+Page.jsx
// Environment: client and/or server

export function Page(pageContext) {
  return <h1>Hello c't webdev</h1>
}
```

Intro
○○○○○

Hooks
●○○○○○

Architecture
○○

Use cases
○○○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
○

```
// pages/+onRenderHtml.js
// Environment: server

import ReactDOMServer from 'react-dom/server'

export function onRenderHtml(pageContext) {
  const { Page } = pageContext
  const pageHtml = ReactDOMServer.renderToString(<Page/>)
  return `
    <html>
      <body>
        <div id="root">${pageHtml}</div>
      </body>
    </html>`
}
```

```
// pages/+onRenderClient.js
// Environment: client

import ReactDOMClient from 'react-dom/client'

export function onRenderClient(pageContext) {
  const { Page } = pageContext
  ReactDOMClient.hydrateRoot(
    document.getElementById('root'),
    <Page/>
  )
}
```

```
// Doesn't depend on Node.js (e.g. can be used in Cloudflare Workers)
import { renderPage } from 'vike/server'

// Any server: Express.js, Cloudflare Worker, AWS Lambda Function, Fastify, Hono, Nitro, ...
server.addMiddleware({
  method: 'GET',
  route: '*', // catch-all
  async handler(request) {
    const pageContextInit = { urlOriginal: request.url }
    const pageContext = await renderPage(pageContextInit)
    // `body` is the HTML of the page with a route matching pageContextInit.urlOriginal
    const { body, statusCode, headers } = pageContext.httpResponse
    const response = { body, statusCode, headers }
    return response
  }
})
```

Intro
○○○○

Hooks
○○○●○○

Architecture
○○

Use cases
○○○

DX
○○○○○○○○○○

Business model
○

Future
○

Conclusion
○


```
// pages/+onCreateGlobalContext.server.js
// Environment: server
export async onCreateGlobalContext() {
  const dataFromAPI = await fetchDataFromAPI()
  const someGlobalData = select(dataFromAPI)
  globalContext.someGlobalData = someGlobalData
}
```

```
// pages/+config.js
export default {
  passToClient: 'someGlobalData'
}
```

```
// Anywhere (client,server)
import { getGlobalContext } from 'vike'
const { someGlobalData } = await getGlobalContext()
```

- `+data.js`
 - Data fetching
- `+onCreateGlobalContext.client.js`
 - Client-side initialization (e.g. authentication)
 - Init store (e.g. Redux or Zustand)
- `+onCreatePageContext.{client,server}.js`
- `+guard.js`
 - Authorization
- `+onPageTransition{Start,End}.js`
 - Full control over page transition animation
- `+onBeforeRoute.js`
 - Full control over i18n integration
- ...

Extensions

- vike-react
- vike-react-rsc 🚧 (Server Components)
- vike-react-redux
- vike-react-zustand
- vike-react-query
- vike-react-apollo
- ...

More: vike-vue, vike-vue-pinia, vike-vue-query, vike-solid, vike-angular, ...

Vike

Stable & flexible core:

- Unopinionated & agnostic $\Rightarrow$ foundational stability
- Low-level hooks $\Rightarrow$ powerful flexibility

Extensions

Powerful extensions:

- Opinionated $\Rightarrow$ pick extensions $\Rightarrow$ assemble *your* dream stack
- Minimal yet deep/full-fledged integration

Extensions are "cheap" (to create) $\Rightarrow$

- Quickly adopts new tech
- Experimental extensions

Intro
○○○○

Hooks
○○○○○○○

Architecture
○●

Use cases
○○○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
○

Use case Ecosia

ecosia.de: Google alternative made in Berlin

- Vue SPA/SSR
- Vue 2 ⇒ Vue 3 progressive migration

Use case Alignable

alignable.com: business owner networking platform

- Backend: Ruby on Rails
- React + SSR
- Mixed: some pages with Ruby, some with Vike

Use case Contra

contra.com: freelance platform (upwork alternative)

- React + SPA/SSR
- Advanced GraphQL Relay integration

Intro
○○○○

Hooks
○○○○○○○

Architecture
○○

Use cases
○●○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
○

Use case Burda

Burda GmbH owns [focus.de](#), [chip.de](#), [bunte.de](#), ...

- Solid (React alternative) + SSR
- **Before:** fragmented stacks
 - ⇒ cannot improve all websites at once
- **Now:** Internal company framework powering all websites
 - ⇒ Software architects control stack of all websites
 - ⇒ App code: product developers
- Vike without extensions
 - ⇒ full control over architecture + foundational stability
- `vike-react` is ~1k LoC ⇒ easy to build your own framework

SPA/SSR/SSG

SSR/SSG:

- HTML streaming
- Progressive rendering (aka partial hydration)

SPA/SSG:

- `+prerender` (boolean) ⇒ disable/enable (on page-by-page basis)
- `+ssr` (boolean) ⇒ toggle SSR/SPA (on page-by-page basis)
- `+data.client.js` ⇒ 100% client-side data fetching
- SSG redirects
- 🚧 SSG dynamic routes (aka SPA fallback)

Easily create SPA like the old days.

⇒ surprisingly hard with other frameworks (but shouldn't!)



Remote Functions. Instead of API.

- RPC (Remote Procedure Call)
- Optional: you can use REST/GraphQL instead
- Simple

Intro
○○○○

Hooks
○○○○○○○

Architecture
○○

Use cases
○○○

DX
○●○○○○○○○

Business model
○

Future
○

Conclusion
○

```

// CreateTodo.telefunc.ts
// Environment: server

// Telefunc makes onNewTodo() remotely callable
// from the browser.
export { onNewTodo }

import { getContext } from 'telefunc'

// Telefunction arguments are automatically validated
// at runtime, so `text` is guaranteed to be a string.
async function onNewTodo(text: string) {
  const { user } = getContext()

  // With an ORM
  await Todo.create({ text, authorId: user.id })

  // With SQL
  await sql(
    'INSERT INTO todo_items VALUES (:text, :authorId)',
    { text, authorId: user.id }
  )
}

```

Server

```

// CreateTodo.tsx
// Environment: client

// CreateTodo.telefunc.ts isn't actually loaded;
// Telefunc transforms it into a thin HTTP client.
import { onNewTodo } from './CreateTodo.telefunc.ts'

async function onClick(form) {
  const text = form.input.value
  // Behind the scenes, Telefunc makes an HTTP request
  // to the server.
  await onNewTodo(text)
}

function CreateTodo() {
  return (
    <form>
      <input input="text"></input>
      <button onClick={onClick}>Add To-Do</button>
    </form>
  )
}

```

Browser

Intro
○○○○

Hooks
○○○○○○○

Architecture
○○

Use cases
○○○

DX
○○●○○○○○

Business model
○

Future
○

Conclusion
○

React Router

```
// article/delete.js
// Only a single action per file 😞
export async function action({ request }) {
  const { article_id } = await request.json()
  await db.article.delete({ article_id })
  return Response.json({ success: true })
}

// Article.jsx
export default function Article() {
  const onClick = async () => {
    // Lots of boilerplate 😞
    const res = await fetch("/article/delete", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ article_id })
    })
    const { success } = await res.json()
    if (!success) alert("Deletion failed")
  }
  return <button onClick={onClick}>Delete</button>
}
```

Vike + Telefunc

```
// Article.telefunc.js
export async function onArticleDelete({ article_id }) {
  await db.article.delete({ article_id })
  return { success: true }
}

// Article.jsx
import { onArticleDelete } from './article.telefunc.js'
export default function Article() {
  const onClick = async () => {
    const { success } = await onArticleDelete({ article_id })
    if (!success) alert("Deletion failed")
  }
  return <button onClick={onClick}>Delete</button>
}
```

Intro
○○○○

Hooks
○○○○○○○

Architecture
○○

Use cases
○○○

DX
○○○●○○○○

Business model
○

Future
○

Conclusion
○

Next.js (app/)

```
// Article.jsx
// Environment: client

import { revalidatePath } from 'next/cache'

export default function Article({ article_id }) {
  async function onArticleDelete() {
    // Environment: server
    'use server'
    // 🙄 Mixed server and client code inside same file
    // 🙄 Magic passing of props
    await db.article.delete({ article_id })
    // 🙄 Complex caching
    revalidatePath('/articles')
  }
  return <button onClick={onArticleDelete}>Delete</button>
}
```

Vike + Telefunc

```
// Article.telefunc.js
// Environment: server

export async function onArticleDelete({ article_id }) {
  await db.article.delete({ article_id })
}
```

```
// Article.jsx
// Environment: client

import { onArticleDelete } from './article.telefunc.js'
export default function Article({ article_id }) {
  const onClick = () => onArticleDelete({ article_id })
  return <button onClick={onClick}>Delete</button>
}
```

✓ Cache is opt-in

Intro
○○○○

Hooks
○○○○○○○

Architecture
○○

Use cases
○○○

DX
○○○○●○○○

Business model
○

Future
○

Conclusion
○

RPC

- Not built-in by design (unlike Remix and Next.js)
- Any RPC tool: Telefunc, tRPC, ...
- Any non-RPC tool: React Query, GraphQL, ...
- Progressive migration from one paradigm to another
 - E.g. progressively migrate from GraphQL to RPC

Intro
○○○○

Hooks
○○○○○○○

Architecture
○○

Use cases
○○○

DX
○○○○○●○○

Business model
○

Future
○

Conclusion
○

npm pnpm yarn bun

>_ npm create vite@latest --- --react --tailwindcss --daisyui --authjs --telefunc --hono --drizzle --eslint

 Try me in Stackblitz

Presets

Frontend

Full-stack

Next.js

Nuxt

CMS

Frontend

Frontend Framework (required)



Vike

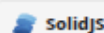
UI Framework (required)



React

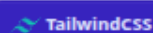


Vue



SolidJS

CSS

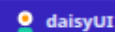


TailwindCSS

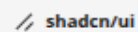


Compiled

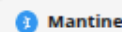
UI Component Libraries



daisyUI



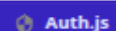
shadcn/ui



Mantine

Data

Auth

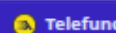


Auth.js



Auth0

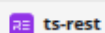
Data fetching



Telefunc

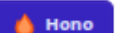


tRPC



ts-rest

Server



Hono



h3

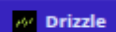


Express

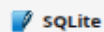


Fastify

Database



Drizzle



SQLite



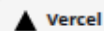
Prisma

Deployment

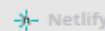
Hosting



Cloudflare



Vercel



Netlify



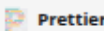
AWS

Utilities

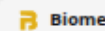
Linter



ESLint



Prettier



Biome

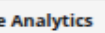
Analytics



Plausible.io

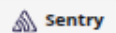


Google Analytics



Segment

Error tracking



Sentry



Logrocket

Details

- Bugs quickly fixed (usually under 24h)
- Deployment skew protection
- Different Base URL for client/server
- `>=400` helpful warnings & error messages
 - Minimal in client-side production to save KBs
- Infinite loop guards
- CSP support
- ...

Open Source Pricing

- 100% MIT licensed
- 100% gratis for engineers
 - No license needed — works like a regular open source tool
- Only larger companies pay
 - Pesky toaster in dev: Purchase a license at ...
 - Only shown for larger teams (Vike uses a heuristic)
 - Pay only as much you can/want

It's a feature ⇒ Vike's interests are aligned with yours.

Intro
○○○○

Hooks
○○○○○○

Architecture
○○

Use cases
○○○

DX
○○○○○○○○

Business model
●

Future
○

Conclusion
○

Future

- Powerful extensions
 - Full-stack extensions — like Django, Ruby on Rails
 - Full-stack default path
 - Partial eject ⇒ progressive & cherry-picked control

Intro
○○○○

Hooks
○○○○○○○

Architecture
○○

Use cases
○○○

DX
○○○○○○○○○

Business model
○

Future
●

Conclusion
○



Vike

Why

- ✓ Architecture (foundational stability & flexibility)
- ✓ Next-generation DX
- ✓ Sustainable business model

Team

- Rom Germany, Munich
- Joël France, Belfort
- Dani Hungary, Lake Balaton
- Muhammad Indonesia, Aceh

Questions?

Intro
○○○○

Hooks
○○○○○○○

Architecture
○○

Use cases
○○○

DX
○○○○○○○○○

Business model
○

Future
○

Conclusion
●